Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

MONOGRAFÍA

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239
Mendez, Jorge Nicolas[1]	DNI 41301342

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,
ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

2026

Índice

Introducción **3**
Objetivos **4**
Desarrollo **5**
Conclusión **6**
Bibliografía **7**

Introducción

La arquitectura del Internet comercial se asienta, en gran medida, en el modelo de solicitud y respuesta: un cliente consulta a un servidor cuando necesita un dato (a través de conexiones efímeras y de mensajes en formatos textuales extensos). Este modelo descansa sobre un presupuesto que rara vez se explicita: que las interacciones son esporádicas, que las inicia una persona y que se realizan sobre enlaces estables, con recursos de cómputo y energía holgados.

Un número creciente de sistemas distribuidos contradice ese presupuesto de manera sistemática. Los dispositivos del "Internet de las Cosas", las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de los firewalls de redes ajenas. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos voluminosos; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, los sistemas distribuidos han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de "Internet de las Cosas") en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para sostener una conexión persistente con sus servidores sin agotar la batería de los teléfonos, lo que permitió, en muchos casos, entregar los mensajes en cientos de milisegundos en lugar de varios segundos. En robótica, InOrbit, una empresa que ofrece una plataforma de orquestación de flotas de robots en la nube, construye su SDK de borde sobre MQTT y sobre mensajes definidos con Protocol Buffers; una decena de brokers desplegados en la nube reciben datos en tiempo real de miles de entidades, que a su vez son controladas por el mismo canal, aun en instalaciones con redes inestables. OpenRobOps, un gestor de flotas de código abierto, publica además los archivos `.proto` que fijan el formato de los datos entre clientes y gestor.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: la entidad ubicada en la red local del cliente es la que inicia la conexión hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

Objetivos

Desarrollo

Conclusión

Bibliografía

Banks, A., Briggs, E., Borgendale, K., & Gupta, R. (eds.). (2019). *MQTT version 5.0* [OASIS Standard]. OASIS. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

Google. (s. f.). *Overview*. Protocol Buffers Documentation. Recuperado 19 de octubre de 2026, de https://protobuf.dev/overview/

InOrbit. (s. f.). *InOrbit.AI: We make robots work*. Recuperado 19 de octubre de 2026, de https://www.inorbit.ai/

inorbit-ai. (s. f.). *edge-sdk-python: InOrbit Python Edge SDK*. GitHub. Recuperado 19 de octubre de 2026, de https://github.com/inorbit-ai/edge-sdk-python

OpenRobOps. (s. f.). *OpenRobOps: Open source robot fleet management*. Recuperado 19 de octubre de 2026, de https://openrobops.org/

OpenRobOps/oro. (s. f.). *oro.proto*. GitHub. Recuperado 19 de octubre de 2026, de https://github.com/OpenRobOps/oro/blob/main/app/public/oro.proto

Zhang, L. (2011, agosto 12). *Building Facebook Messenger*. Engineering at Meta. https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/